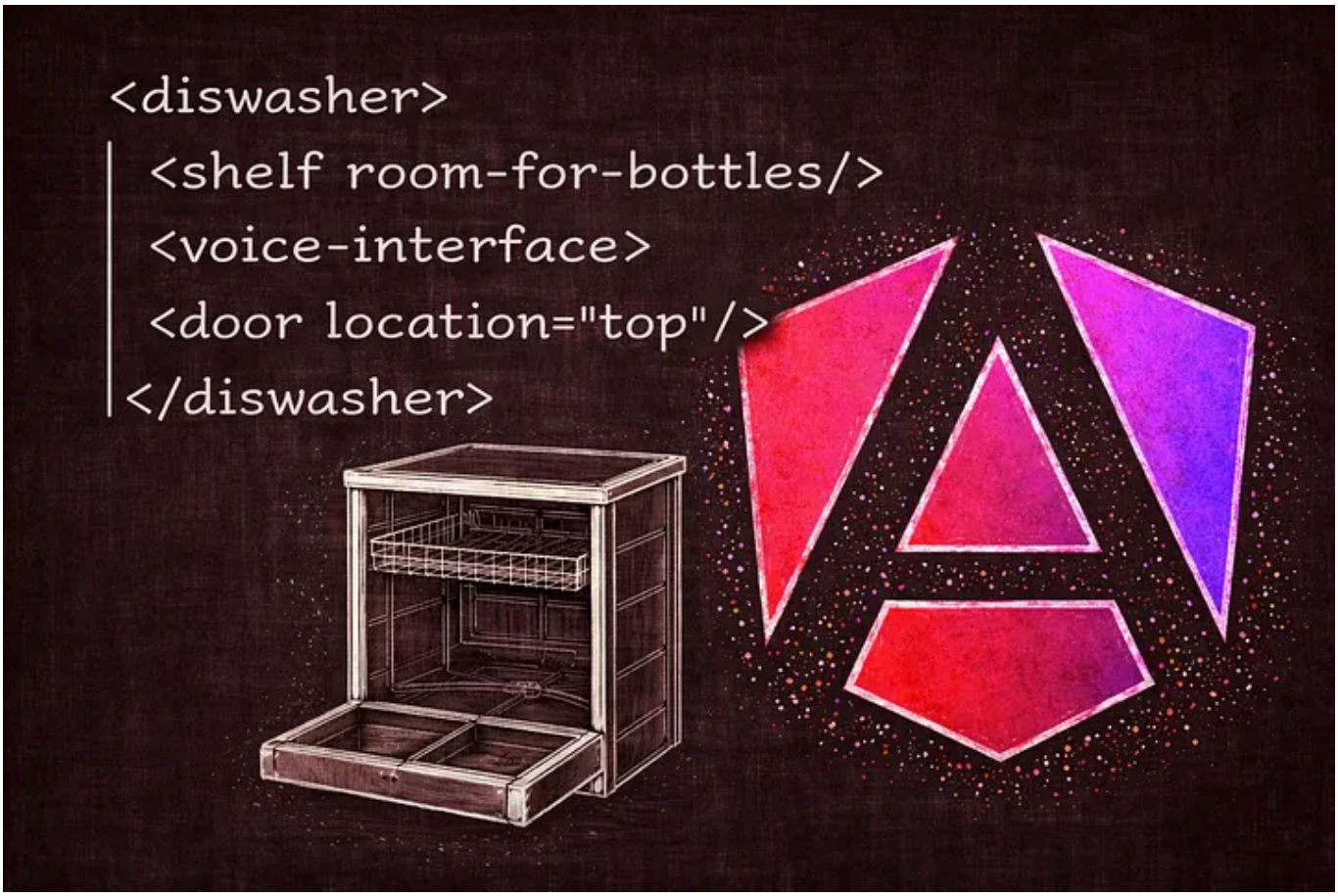
A Better way to build Angular Components: From Inputs to Composition

Building Scalable UI with the Composite Components Pattern

Kobi Hari Follow 4 min read · 5 days ago

82



You are building a reusable component. You start with a simple, elegant component — maybe a basic expander — that fits a single use case. But as requirements grow, so does the component.

You add more and more inputs, and before you know it, you've created a "**monster**" — a complex piece of code packed with features, customization options, and countless inputs.

There's a better way.

The Trap of "Input Overload"

When we want to add a new feature, the most common instinct is to add another `input()` . For example, if you want your expander's header to be clickable to toggle the content, you might add

```
@Component({
  ...
  selector: 'app-expander'
  ...
})
export class ExpanderComponent {
  ...
  readonly useHeaderAsToggle = input(false);
  ...
}
```

However, this approach quickly leads to several problems:

- **Maintainability Nightmares:** You end up with components that are 1,000 lines long, filled with if statements and **spaghetti code** that is nearly impossible to understand. Even for this simple example you need conditional styling for clickable headers, conditional event handlers, and perhaps conditional markup.
- **Unreadable HTML:** If your component supports 50 features, the consumer might have to manage 50 different inputs in their template, making the HTML impossible to read.
- **Poor Logic Separation:** Features that belong to a sub-part (like the header) are being managed by the parent (the expander), which doesn't make architectural sense.

To demonstrate the last point — look at this code:

```
<app-expander [useHeaderAsToggle]="true">
  <span expander-header>The Composite Components Pattern</span>
</app-expander>
```

The behavior applies to the **header**…
but the configuration sits on the **parent**.

So when reading the template, you're forced to "jump" between elements to understand what's going on.

That's a design smell.

What are Composite Components?

The **Composite Components** pattern shifts the focus from "packing features in" to **composing features together**. The idea is simple:

- The main component stays small
- Features live in separate units (directives / sub-components)
- Consumers combine them as needed

Practical Example: The Expander Header Toggle

Instead of adding another input, we move the behavior into a directive.

```
@Directive({
  selector: '[expander-header][toggle]', // Requires both attributes to apply
  host: {
      '[style.cursor]': '"pointer"',
      '(click)': 'onClick()'
  }
})
export class ExpanderHeaderToggleDirective {
  // Inject the parent expander component to access its logic
  private expander = inject(ExpanderComponent, { optional: true });


  // Handle the click event to call the expander's toggle method
  onClick() {
    this.expander?.toggle();
  }
}
```

Usage Becomes:

```
<app-expander>
  <span expander-header toggle>The Composite Components Pattern</span>
</app-expander>
```

What Did We Gain?

- No new inputs
- No changes to the main component
- Feature is fully encapsulated
- The expander stays clean and focused.

Real-World Inspiration: Angular Material

This isn't a new or niche pattern; it has been used by **Angular Material** since its inception.

- **`<mat-form-field>`** : This component doesn't have a massive list of inputs for labels, hints, or icons. Instead, you **compose** it using directives like `matInput`, `<mat-label>`, and `matSuffix`.
- **`<mat-card>`** : A card is a package of sub-components like `<mat-card-header>` and directives like `matCardAvater` . This allows the main card component to have very few inputs while remaining incredibly flexible.

2. **Flexibility:** Users can mix and match exactly which features they need for a specific instance.
3. **Clean APIs:** Your main component doesn't get cluttered with irrelevant inputs.
4. **Standardization:** This pattern is becoming the standard for modern component libraries and is heavily utilized in newer packages like **Angular Aria** (often paired with headless component patterns).

By adopting composite components, you can build a robust library where features **play nicely together** without turning your codebase into a monster. In the long run, this allows you to maintain the same codebase across many different projects with ease.

What's next

If this resonates, I go much deeper into this pattern — including advanced composition techniques and real-world examples — in my new Udemy course, *Building Reusable Components in Angular — The Missing Guide.*

Building Reusable Components in Angular — The Missing Guide

It's a hands-on deep dive into composable components, content projection, templates, and the patterns behind real-world component libraries.

Angular | Web Components | Composition

Written by Kobi Hari

118 followers · 5 following

I love programming and learning new technologies and I love teaching it. I'm the author of the "**Angular Missing Guide**" series on **Udemy**

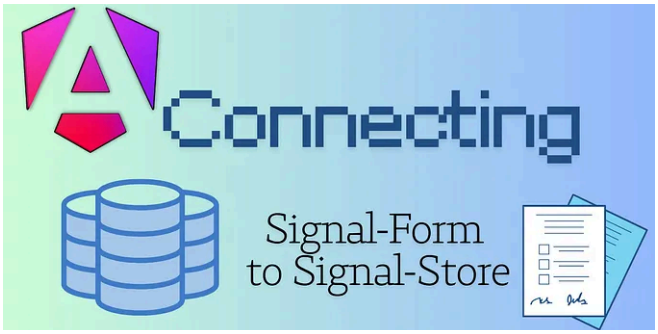
Follow

No responses yet

Write a response

What are your thoughts?

More from Kobi Hari



Kobi Hari

Connecting Signal Forms to a Signal Store

In my courses, I get this question a lot:

Dec 31, 2025 43 1

Kobi Hari

Reusable Validation in Angular's Signal Forms

I Hate repeating myself. Again... I Hate repeating myself. Especially in code.

Feb 1 26 1

Kobi Hari

The Most Exciting Feature of Angular Signal Forms No One...

It Started with a Weird Compilation Error

Nov 20, 2025 351 3

Kobi Hari

How Did I Miss This After 5 Years: TypeScript's String Literal...

A Simple Search for a UID Turns Into Something Else

Dec 8, 2025 24 2

See all from Kobi Hari

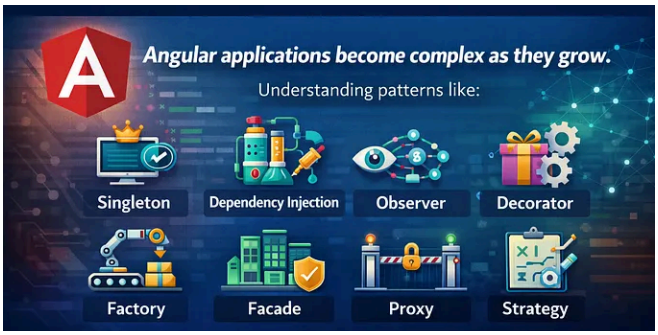
Recommended from Medium

In Level Up Coding by Pawan Kumawat

How to Optimize Your Use of Angular Signals

Angular Signals give you fine-grained reactivity and better performance — if you...

Mar 30 177



Sumit Ranjan

Angular Design Patterns Explained: Real-World Examples...

Explained with Real-World Examples

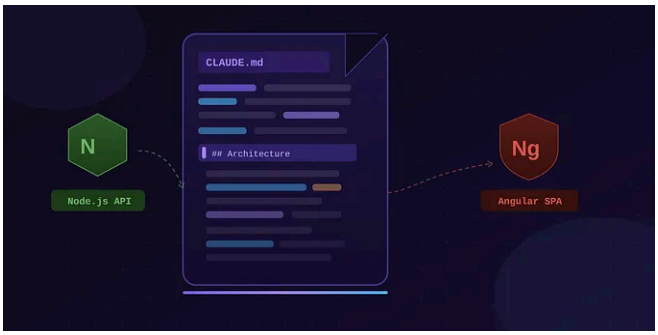
Mar 9 8 1

In JavaScript in Plain English by CodePulse

State Management Without Boilerplate: The "Signal Store"...

Redux was fantastic in 2018. In 2026, you don't need Actions, Reducers, Effects, and...

Mar 30 259 1



Satnam Singh

CLAUDE.md Best Practices: Building a Smarter AI-Assisted...

Give Claude a map of your codebase — once — and watch it work like a senior team...

Mar 29 6

Yogesh Raghav

⚠️ Angular Signals in Production — What Broke After 3 Months (Re...

Three months ago, we migrated a major feature module to Angular Signals.

Feb 21 127 3

In SkillStuff by CodeByUmar

Don’t Start Another Node.js Project Without These 5 Packages

Deep dive into small tools that quietly eliminate real backend pain points

Mar 30 120 2

See more recommendations